<agg-func>(<column_name>) over ()

-- running total

```
select *,
sum(salary) over (order by hire_date) as total_sal
from employees;

CREATE TABLE sales (
sale_date DATE,
product_id INT,
product_name VARCHAR(100),
units_sold INT
);

INSERT INTO sales (sale_date, product_id, product_name, units_sold)
VALUES
('2024-08-15', 1, 'Laptop', 4),
('2024-08-16', 1, 'Laptop', 6),
('2024-08-17', 1, 'Laptop', 5),
('2024-08-18', 1, 'Laptop', 7),
('2024-08-19', 1, 'Laptop', 4),
('2024-08-20', 1, 'Laptop', 5),
('2024-08-21', 1, 'Laptop', 8),
('2024-08-22', 1, 'Laptop', 3),
('2024-08-23', 1, 'Laptop', 7),
('2024-08-24', 1, 'Laptop', 6),
('2024-08-25', 1, 'Laptop', 9),
('2024-08-26', 1, 'Laptop', 7),

('2024-08-15', 2, 'Smartphone', 15),
('2024-08-16', 2, 'Smartphone', 18),
('2024-08-17', 2, 'Smartphone', 12),
('2024-08-18', 2, 'Smartphone', 20),
('2024-08-19', 2, 'Smartphone', 17),
('2024-08-20', 2, 'Smartphone', 14),
('2024-08-21', 2, 'Smartphone', 19),
('2024-08-22', 2, 'Smartphone', 13),
('2024-08-23', 2, 'Smartphone', 21),
('2024-08-24', 2, 'Smartphone', 16),
('2024-08-25', 2, 'Smartphone', 18),
('2024-08-26', 2, 'Smartphone', 20),

('2024-08-15', 3, 'Tablet', 8),
('2024-08-16', 3, 'Tablet', 10),
('2024-08-17', 3, 'Tablet', 7),
('2024-08-18', 3, 'Tablet', 12),
('2024-08-19', 3, 'Tablet', 9),
('2024-08-20', 3, 'Tablet', 8),
```

```
('2024-08-21', 3, 'Tablet', 11),
('2024-08-22', 3, 'Tablet', 6),
('2024-08-23', 3, 'Tablet', 9),
('2024-08-24', 3, 'Tablet', 10),
('2024-08-25', 3, 'Tablet', 13),
('2024-08-26', 3, 'Tablet', 12);

-- running total for each product

select *,
sum(units_sold) over (partition by product_id order by sale_date)
as running_total
from sales;

-- what if I remove partition by
-- in case of duplicates it consider it as one group

select *,
sum(units_sold) over (order by sale_date)
as running_total
from sales;

-- you can always give a second column in order to remove the ties.
select *,
sum(units_sold) over (order by sale_date,product_id)
as running_total
from sales;


select *,
sum(units_sold) over (partition by product_id order by sale_date)
as running_total
from sales;

-- what if I need to have a moving window of 3 days

select *,
sum(units_sold) over (partition by product_id order by sale_date
rows between 2 preceding and current row)
from sales;

select *,
sum(units_sold) over (partition by product_id order by sale_date desc
rows between current row and 2 following) as 3_day_sales
from sales;
```

20
19
18
17
16
15

what if I say

```
select *,
sum(units_sold) over (partition by product_id order by sale_date
rows between 2 preceding and 2 following) as 5_day_sales
from sales;

select *,
sum(units_sold) over (partition by product_id order by sale_date
rows between unbounded preceding and current row) as 5_day_sales
from sales;

select *,
sum(units_sold) over (partition by product_id order by sale_date
rows between unbounded preceding and current row) as running_total1,
sum(units_sold) over (partition by product_id order by sale_date)
as running_total2
from sales;


-- this is the default behaviour also

select *,
sum(units_sold) over (partition by product_id order by sale_date desc
rows between current row and unbounded following) as running_total
from sales;

select *,
sum(units_sold) over (partition by product_id order by sale_date desc
rows between unbounded preceding and unbounded following) as
running_total
from sales;

select *,
sum(units_sold) over (partition by product_id order by sale_date desc
rows between unbounded preceding and unbounded following) as
running_total1,
sum(units_sold) over (partition by product_id) as running_total2
from sales;
```

sum(salary) over (<partition by> <order by>)

rows between

current row
2 preceding
2 following
unbounded preceding
unbounded following